

### Instructions

- Assignments must be written in hand.
- Write the programming in Python.
- Submit before or on the final exam day.

#### 1. Write a program that asks for a user's birth year and calculates their current age.

```
from datetime import datetime
birth_year = int(input("Enter your birth year: "))
current_year = datetime.now().year
age = current_year - birth_year
print("Your current age is:", age)
```

#### 2. Write a program that takes a year as input and determines if it is a leap year.

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a leap year.")
else:
    print(year, "is not a leap year.")
```

#### 3. Write a program that prints the first 10 numbers of the Fibonacci sequence using a loop.

```
n = 10
a = 0
b = 1
print("Fibonacci sequence:")
for i in range(n):
    print(a, end=" ")
    temp = a + b
    a = b
    b = temp
```

#### 4. Problem: Given a list of unsorted integers, find the largest number without using built-in functions.

```
def find_max(numbers):
    maximum = numbers[0]
    for num in numbers:
        if num > maximum:
            maximum = num
    return maximum
```

```
# start here
arr = [10, 45, 2, 89, 33, 100, 67]
```

```
result = find_max(arr)
print("Maximum value is:", result)
```

**5. Write a function that takes a string and returns a dictionary or map showing how many times each character appears.**

```
def char_count(text):
    text = text.lower()
    result = {}
    for char in text:
        if char in result:
            result[char] += 1
        else:
            result[char] = 1
    return result
```

```
# Example usage
string = input("Enter a string: ")
print(char_count(string))
```

**6. Define a Student class with attributes for name, ID, and a list of grades, including a method to calculate the average grade.**

```
class Student:
    def __init__(self, name, student_id, grades=None):
        self.name = name
        self.student_id = student_id
        self.grades = grades if grades is not None else []
    def add_grade(self, grade):
        self.grades.append(grade)
    def calculate_average(self):
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)
```

```
# Example usage
student = Student("Rahim", "S101", [80, 85, 90])
print("Name:", student.name)
print("ID:", student.student_id)
student.add_grade(95)
```

```
print("Grades:", student.grades)

print("Average:", student.calculate_average())
```

### **7. Implement a linear search function that finds and returns the index of a target name in a list of names.**

```
def linear_search(names, target):
    for index, name in enumerate(names):
        if name == target:
            return index
    return -1
```

# Example usage

```
names = ["Rahim", "Karim", "Habib", "Ayesha"]
result = linear_search(names, "Habib")
print(result) # Output: 2
```

### **8. Implement the selection sort algorithm to sort an array of integers in ascending order.**

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # swap
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
```

# Example

```
numbers = [10, 20, 3, 40, 2, 1, 90]
sorted_numbers = selection_sort(numbers)
print(sorted_numbers)
```

### **9. Write a program using a stack data structure to check if a string of mathematical parentheses is perfectly balanced.**

```

def is_valid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'}
    for char in s:
        if char in pairs.values():
            stack.append(char)
        elif char in pairs.keys():
            if not stack or stack.pop() != pairs[char]:
                return False
    return len(stack) == 0

```

```

# ♦ Test the function
s = input("Enter brackets string: ")
if is_valid(s):
    print("Valid Parentheses")
else:
    print("Invalid Parentheses")

```

**10. Write a program that simulates memory allocation by finding contiguous available blocks in a boolean array and marking them as used.**

```

def allocate_memory(memory, size):
    n = len(memory)
    for i in range(n - size + 1):
        can_allocate = True
        # check continuous block
        for j in range(i, i + size):
            if memory[j] == True:
                can_allocate = False
                break
        # if found free block
        if can_allocate:
            for j in range(i, i + size):

```

```
memory[j] = True
```

```
# Example usage
```

```
memory = [False, True, False, False]
```

```
size = 2
```

```
index = allocate_memory(memory, size)
```

```
print("Updated memory:", memory)
```

### 11. Draw the recursion tree for calculating the 5<sup>th</sup> Fibonacci number and identify the overlapping subproblems.

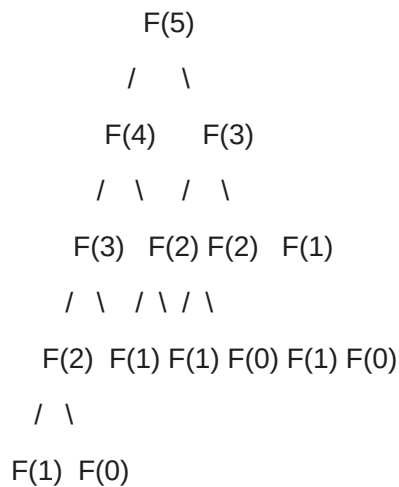
The Fibonacci sequence is defined using a recursive relation where each number is the sum of the previous two numbers.

$$F(n) = F(n-1) + F(n-2)$$

Where,

$F(0) = 0$  and  $F(1) = 1$  are the base cases.

To calculate Fibonacci(5), the recursive function expands as follows:



### Overlapping Subproblem

In Fibonacci(5), the unique subproblems are:

$F(0), F(1), F(2), F(3), F(4), F(5) \rightarrow$  total **6 unique subproblems**

However, from the recursion tree, we can see that:

- $F(3)$  is calculated multiple times
- $F(2)$  is calculated multiple times
- $F(1)$  and  $F(0)$  are also repeated

Although there are only **6 unique subproblems**, the total number of function calls grows exponentially (approximately  $O(2^n)$ ).

Therefore, the same subproblems are solved repeatedly, which indicates the presence of **overlapping subproblems**.

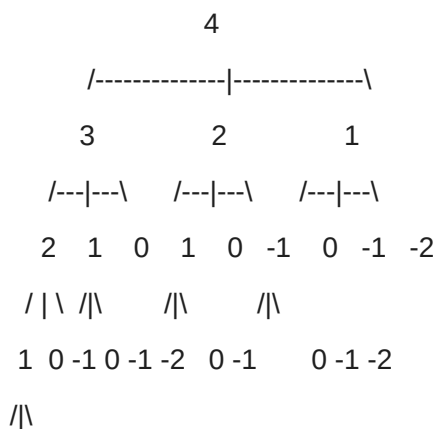
**12. Draw the recursion tree showing all possible paths to make change for the amount 4 using coin denominations of 1, 2, and 3.**

Concept

In this problem, we start from the target amount **4** and at each step we subtract one of the available coins (1, 2, or 3).

Each node in the recursion tree represents the **remaining amount**.

- If the value becomes **0**, it represents a valid solution ✓
- If the value becomes **negative**, it represents an invalid path ✗



0 -1 -2

## Explanation

- We start with the initial amount **4**.
- From each node, we subtract:
  - 1 coin → left branch
  - 2 coins → middle branch
  - 3 coins → right branch
- This process continues until:
  - We reach **0 (valid solution)**
  - Or a **negative value (invalid path)**

**13. Draw the recursion tree for the possible paths a robot can take to reach the bottom-right corner of a 3x3 grid, moving only right and down.**

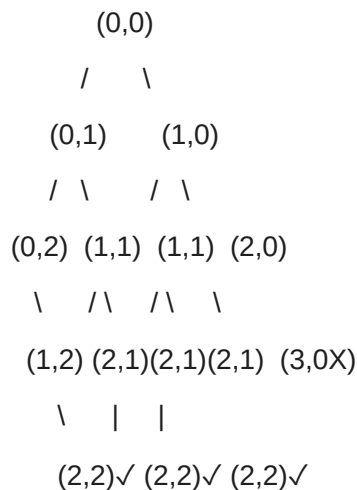
## Concept

At each position, the robot has two choices:

- Move Right → increases column index
- Move Down → increases row index

The process continues until the robot reaches the destination **(2,2)**.

Each node represents the **current position of the robot**.



**14. Trace the step-by-step execution flow and variable states for a loop that determines if the number 7 is prime.**

A number is called **prime** if it is divisible only by 1 and itself.

In this problem, we check whether the number 7 is prime by testing divisibility from 2 to  $n-1$ .

| Iter | i | Condition ( $7 \% i == 0$ ) | Result        | Flag (Prime?) |
|------|---|-----------------------------|---------------|---------------|
| 1    | 2 | $7 \% 2 \neq 0$             | Not divisible | True          |
| 2    | 3 | $7 \% 3 \neq 0$             | Not divisible | True          |
| 3    | 4 | $7 \% 4 \neq 0$             | Not divisible | True          |
| 4    | 5 | $7 \% 5 \neq 0$             | Not divisible | True          |
| 5    | 6 | $7 \% 6 \neq 0$             | Not divisible | True          |

**15. Track the step-by-step execution path, including low, mid, and high index values, when performing a binary search for the number 14 in the sorted array [3, 8, 14, 22, 35, 41, 56, 78].**

Binary Search is an efficient searching algorithm used on a **sorted array**. It works by repeatedly dividing the search range in half and comparing the middle element with the target value.

Here, we track the steps to find **14** in the array:

[3, 8, 14, 22, 35, 41, 56, 78]

| Iter | Low | High | Mid | Value | Logic Check          | Next Action          |
|------|-----|------|-----|-------|----------------------|----------------------|
| 1    | 0   | 7    | 3   | 22    | $22 > 14$ (Too Big)  | $high = mid - 1 = 2$ |
| 2    | 0   | 2    | 1   | 8     | $8 < 14$ (Too Small) | $low = mid + 1 = 2$  |
| 3    | 2   | 2    | 2   | 14    | $14 == 14$ (Match)   | Break                |

Given Array list:

[3, 8, 14, 22, 35, 41, 56, 78]

**def binary\_search(arr, target):**

**low = 0**

```

high = len(arr) - 1
while low <= high:
    mid = (low + high)
    if arr[mid] == target:
        return True
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return False

# Only pass the array here
arr = [3, 8, 14, 22, 35, 41, 56, 78]
target = 14
if binary_search(arr, target):
    print("FOUND ✔")
else:
    print("NOT FOUND ✖")

```

| Iter | Low | High | Mid | Value at Mid | Logic Check | Next Action        |
|------|-----|------|-----|--------------|-------------|--------------------|
| 1    | 0   | 7    | 3   | 22           | 14 < 22     | high = mid - 1 = 2 |
| 2    | 0   | 2    | 1   | 8            | 14 > 8      | low = mid + 1 = 2  |
| 3    | 2   | 2    | 2   | 14           | 14 == 14    | FOUND ✔            |